

Synchronization and Await Statement

- We learned that the execution order of statements in different threads matters. To avoid race conditions while keeping parallelism, we learned to use atomic regions to avoid interleaving the critical section in a thread; it is not enough, we often want threads to synchronize: we want one thread to wait until some other thread makes a condition come true.

Let's look at the following example.

- In the following parallel program, the calculation of u doesn't start until we finish calculating z , even though u doesn't depend on z ; this program can have "more parallelism".

$$[x := \dots \parallel y := \dots \parallel z := \dots]; u := f(x, y)$$

On the other hand, we cannot nest parallel programs, so it is not legal to write:

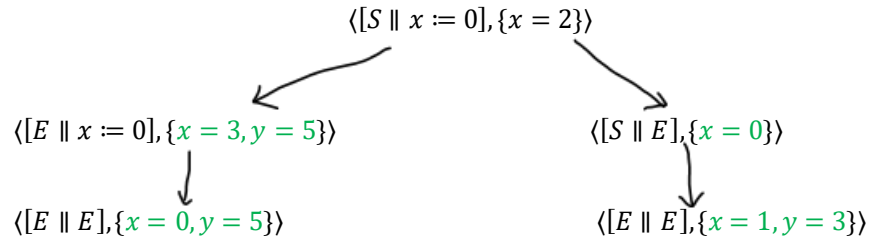
$$[[x := \dots \parallel y := \dots]; u := f(x, y) \parallel z := \dots]$$

It is natural that we want u and z being calculated at the same time, but we need to wait for x and y in the thread of calculating u . We need something like:

$$[x := \dots \parallel y := \dots \parallel \text{wait for } x \text{ and } y, \text{ then } u := f(x, y) \parallel z := \dots]$$

- Here we introduce the await statement: **await B then S end**, where B is a Boolean expression and S is a statement.
 - await** statements can only appear in threads of parallel programs, and they cannot appear inside of atomic regions.
 - Each **await** statement itself is an atomic region: once B is evaluated to true, the execution of S is done in one step.
 - S cannot contain loops, **await** statements, or atomic regions.
 - Not allowing loops in an **await** statement is for the program designing reason. Loops are usually the important part of a program; we don't want the important part of the program to be potentially blocked forever.
- An **await** is like an atomic **if – then** statement, but not identical.
 - With **< if B then S fi >**; if B is false, we execute **skip** and complete the **if – fi**.
 - With **await B then S end**; if B is false, then this await statement **blocks** (no execution in this thread) until B becomes true.
- Here is the semantic of parallel programs with await statements.
 - We still randomly choose between all the available threads. In other words, there's no insistence that we must check the **await** before trying other threads.
 - For example, it is possible that $\langle [\text{await } x \geq 0 \text{ then } x := x + x \text{ end} \parallel x := -1], \{x = 2\} \rangle \rightarrow \langle [\text{await } x \geq 0 \text{ then } x := x + x \text{ end} \parallel E], \{x = -1\} \rangle$.
 - If we choose the thread that begins with **await B then S end**, and B is true, then immediately jump to S and execute all of it.
 - For example, in $\langle [\text{await } x \geq 0 \text{ then } x := x + x; x := x + 1 \text{ end} \parallel x := -1], \{x = 2\} \rangle \rightarrow \langle [E \parallel x := -1], \{x = 5\} \rangle$, the first thread is chosen to be execute first, and there is no chance for $x := -1$ to jump in after $x \geq 0$ is evaluated and before the whole await statement is executed.

- If we choose the thread that begins with **await B then S end**, and B is false, then this await statement blocks. Instead, we randomly choose among other threads to execute.
 - For example, $\langle [\text{await } x = 1 \text{ then } S \text{ end} \parallel x := 1], \{x = 0\} \rangle \rightarrow \langle [\text{await } x = 1 \text{ then } S \text{ end} \parallel E], \{x = 0\} \rangle$ and this is the *only* possible next evaluation step since the first thread blocks until $x = 1$ becomes true.
- 2. Show the evaluation graph of $\langle [S \parallel x := 0], \{x = 2\} \rangle$, where $S \equiv \text{await } x \geq 0 \text{ then } x := x + 1; y := x + 2 \text{ end}$.



- 3. Let's consider some special situations in the await statement **await B then S end**.
 - a. What if $B \equiv T$?
The await statement never blocks and always executes S atomically, thus we can optimize **await T then S end** to $\langle S \rangle$.
 - b. What if $S \equiv \text{skip}$?
[Definition] **wait B** $\equiv$ **await B then skip end**.
 - c. What if $B \equiv F$?
The await statement **blocks forever**.
- 4. What is the difference between **wait B; < S >** and **await B then S end**?
 - With **await B then S end**, once B is true, we immediately atomically execute S , so no other statement can be executed between the test B and running S .
 - With **wait B; < S >**, it allows another thread to be executed after the **wait** and before executing S .

Rules and Triples for Await Statement

- **Await Rule:**
 1. $\{p \wedge B\} S \{q\}$
 2. $\{p\} \text{await } B \text{ then } S \text{ end} \{q\}$ await 1
 - The await rule is very similar to conditional rule 1 with only the true branch.
- We represent await rule in the proof outline as follows:

$$\{p\} \text{await } B \text{ then } \{p \wedge B\} S \{q\} \text{ end } \{q\}$$
- 5. How to understand the await rule for $\{p\} \text{await } F \text{ then } S \text{ end} \{q\}$?
 - Since $\{p \wedge F\} S \{q\}$ is valid (under either correctness), so $\{p\} \text{await } F \text{ then } S \text{ end} \{q\}$ is also valid. This sounds against our intuition, but $\{p\} \text{await } F \text{ then } S \text{ end} \{q\}$ being valid doesn't mean it won't block; it

simply means “if precondition p is satisfied and if the await condition B is satisfied, then the execution of S will satisfy the postcondition q .”

- $wp(\text{await } B \text{ then } S \text{ end}, q) \equiv D(B) \wedge (B \rightarrow wp(S, q))$
- $sp(p, \text{await } B \text{ then } S \text{ end}) \equiv sp(p \wedge B, S)$